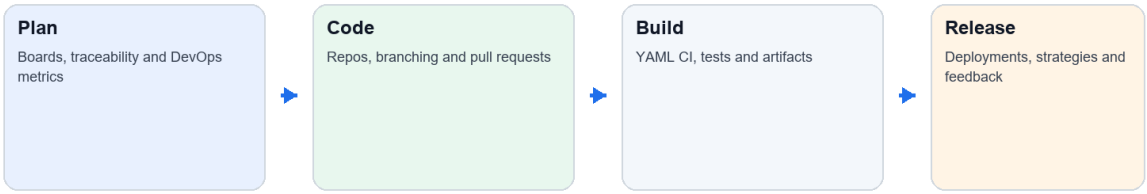
LEARNER GUIDE

Designing and Implementing Microsoft DevOps Solutions (AZ-400)

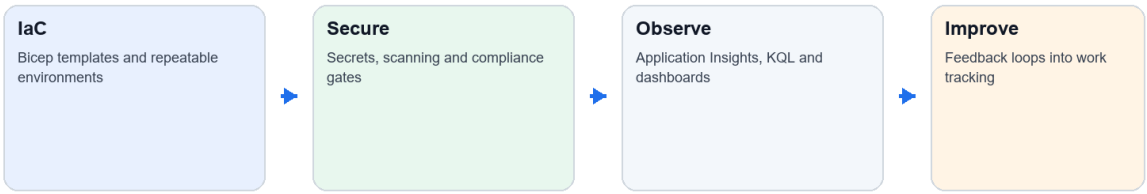
TGS-2024042605 | Version 1 | Tertiary Infotech Academy Pte Ltd

AZ-400 DevOps Delivery Flow



Single source: 8 lab files drive the PPT, Learner Guide, Lesson Plan and Markdown mirror.

Secure DevOps Governance Loop



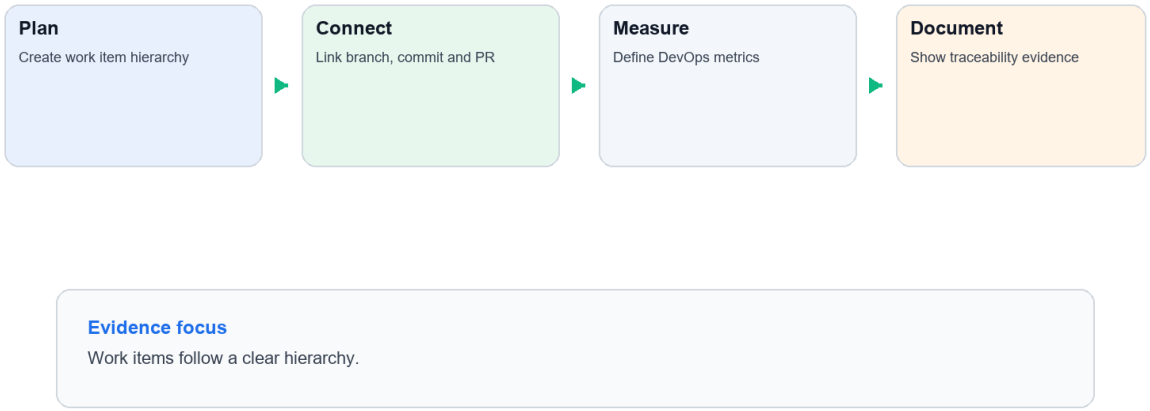
Single source: 8 lab files drive the PPT, Learner Guide, Lesson Plan and Markdown mirror.

Lab 01 - DevOps Processes, Boards, and Traceability

Source: courseware/labs-source/lab-01-devops-processes-boards-traceability.md

Lab 01: DevOps Processes, Boards, and Traceability

Visual workflow map



What you will produce: Boards → Branch → Commit → Pull Request → Dashboard

Objectives

- Create a work tracking structure.
- Link work to code and releases.
- Define DevOps metrics.
- Create project documentation.

Step-by-Step Guide

1. Create or open an Azure DevOps project or GitHub repository.
2. Create sample epics, features, user stories, tasks, and bugs.
3. Define workflow states.
4. Add labels for priority, security, quality, and release.
5. Link a work item to a branch, commit, and pull request.
6. Create a dashboard or metric table.
7. Define lead time, cycle time, deployment frequency, change failure rate, and MTTR.
8. Create Markdown documentation.
9. Add a Mermaid flow diagram.
10. Document traceability from requirement to deployment.

Validation

- Work items follow a clear hierarchy.
- Traceability includes work, code, pipeline, and release evidence.
- Metrics are defined.
- Documentation is readable.

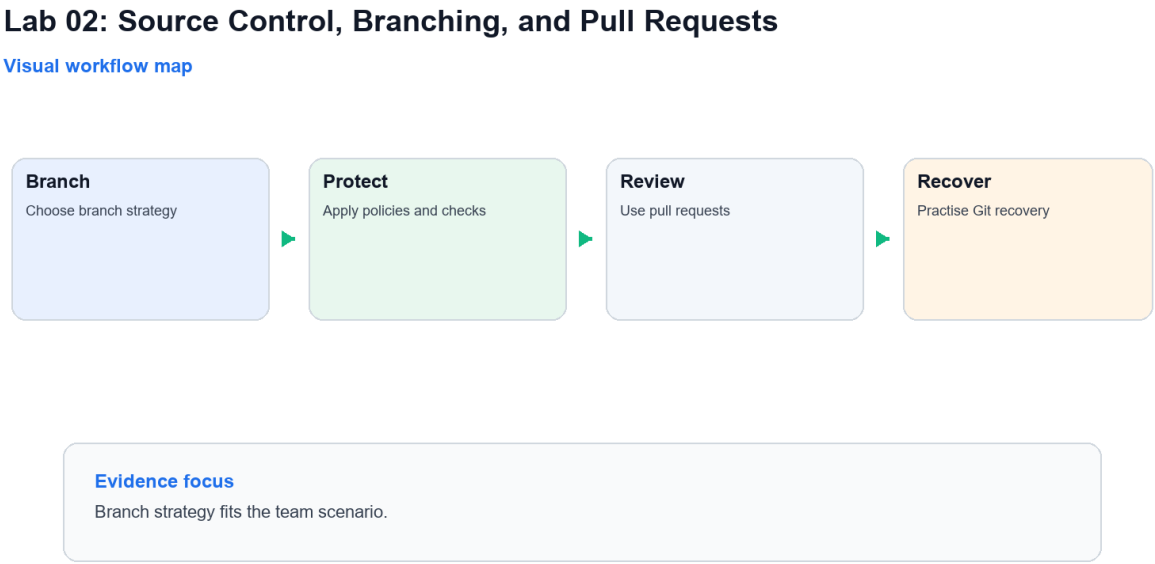
Review Questions

1. Why does traceability matter?
2. What is the difference between lead time and cycle time?

3. How can dashboards improve feedback?
4. Why document architecture and releases?

Lab 02 - Source Control, Branching, and Pull Requests

Source: courseware/labs-source/lab-02-source-control-branching-pull-requests.md



What you will produce: Repository → Branch → Pull Request → Review

Objectives

- Choose a branch strategy.
- Configure pull request workflow.
- Apply branch protection or policies.
- Practise Git recovery.

Step-by-Step Guide

1. Create or open a Git repository.
2. Compare trunk-based, feature branch, and release branch strategies.
3. Choose a branch strategy and document why.
4. Create a feature branch.
5. Make a small change.
6. Commit and push the branch.
7. Open a pull request.
8. Require review and status checks where available.
9. Add a release tag.
10. Recover a previous file version with Git.
11. Document how sensitive data removal should be escalated.

Validation

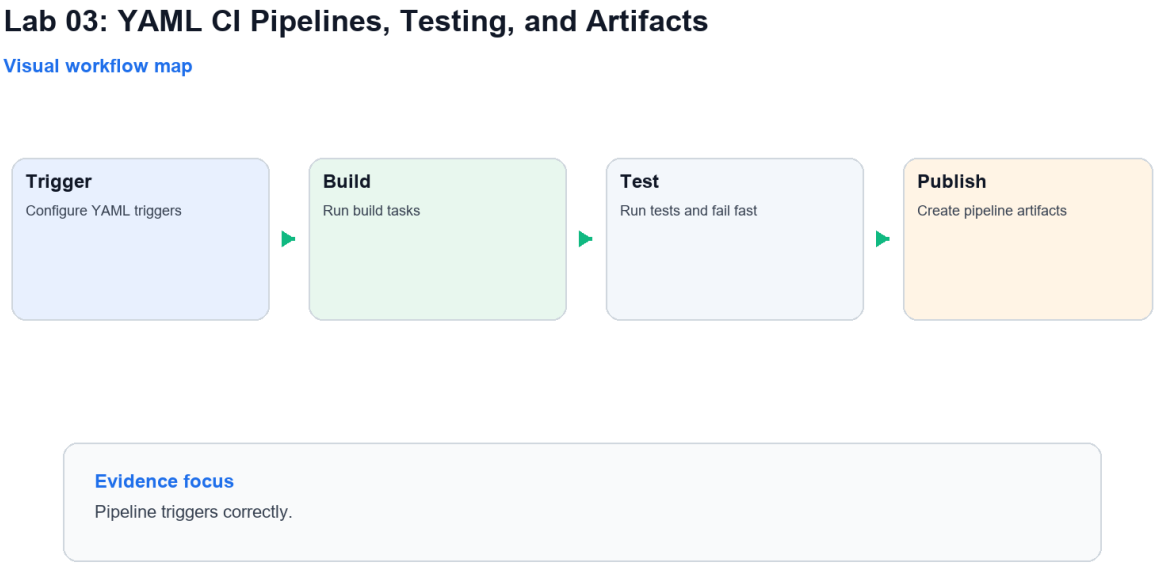
- Branch strategy fits the team scenario.
- Pull request includes review and checks.
- Tagging convention is documented.
- Git recovery step is successful.

Review Questions

1. When is trunk-based development useful?
2. Why protect main?
3. What is the purpose of release tags?
4. Why is secret removal from Git history sensitive?

Lab 03 - YAML CI Pipelines, Testing, and Artifacts

Source: courseware/labs-source/lab-03-yaml-ci-pipelines-testing-artifacts.md



What you will produce: YAML Pipeline → Build → Test → Artifact

Objectives

- Create a YAML CI pipeline.
- Configure triggers.
- Run tests.
- Publish artifacts.

Step-by-Step Guide

1. Use a sample application repository.
2. Add a YAML pipeline file.
3. Configure triggers for main and pull requests.
4. Add build steps.
5. Add a unit test step.
6. Publish test results where supported.
7. Publish a build artifact.
8. Use variables or reusable steps.
9. Run the pipeline.
10. Inspect logs.
11. Fix one controlled failure.
12. Record duration and improvement ideas.

Validation

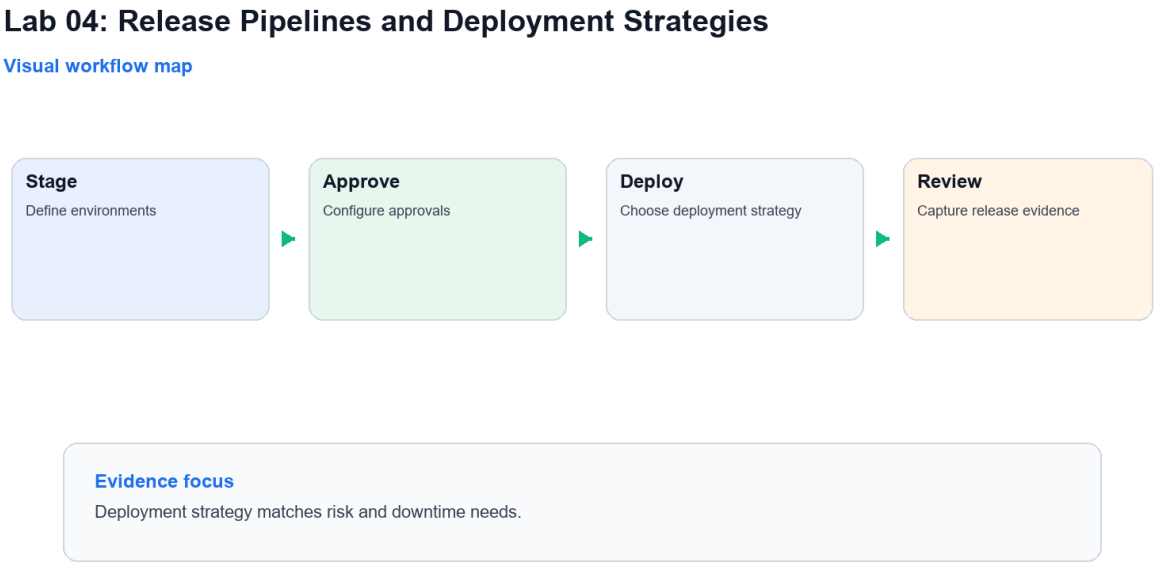
- Pipeline triggers correctly.
- Tests run.
- Artifact is published.
- Failure is documented and fixed.

Review Questions

1. Why define pipelines as YAML?
2. What should happen on pull request builds?
3. Why publish artifacts?
4. What makes a test flaky?

Lab 04 - Release Pipelines and Deployment Strategies

Source: courseware/labs-source/lab-04-release-pipelines-deployment-strategies.md



What you will produce: Artifact → Environment → Approval → Deployment

Objectives

- Design release stages.
- Configure environments and approvals.
- Compare deployment strategies.
- Plan rollback and hotfix paths.

Step-by-Step Guide

1. Define dev, test, staging, and production environments.
2. Add deployment stages to a pipeline.
3. Configure approvals or checks for higher environments.
4. Compare blue-green, canary, ring, rolling, feature flags, and deployment slots.
5. Choose a strategy for a sample app.
6. Deploy a sample artifact or simulate deployment.
7. Define rollback criteria.
8. Define rollback steps.
9. Define a hotfix path.
10. Create release notes from commits or PRs.

Validation

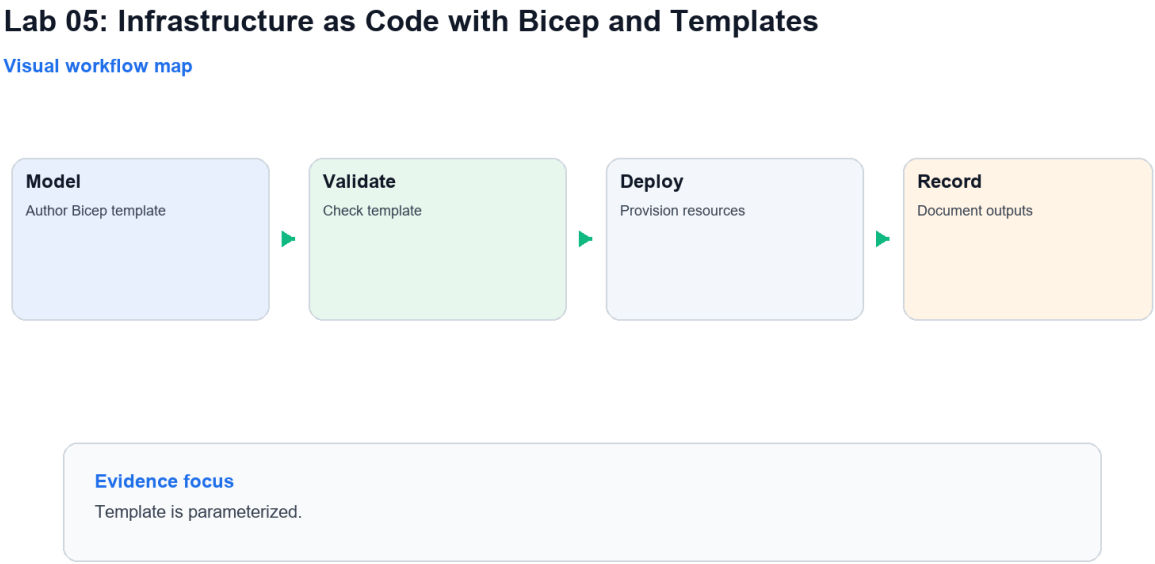
- Deployment strategy matches risk and downtime needs.
- Approval/check evidence is captured.
- Rollback path is clear.
- Release notes are generated or drafted.

Review Questions

1. When would you use canary deployment?
2. Why are approvals useful?
3. What should a rollback plan include?
4. How do feature flags reduce deployment risk?

Lab 05 - Infrastructure as Code with Bicep and Templates

Source: courseware/labs-source/lab-05-infrastructure-as-code-bicep-templates.md



What you will produce: Bicep Template → Validation → Deployment

Objectives

- Define IaC strategy.
- Create a Bicep or template file.
- Validate deployment.
- Document review and drift controls.

Step-by-Step Guide

1. Identify infrastructure required by the sample app.
2. Create or review a Bicep template.
3. Add parameters for environment and location.
4. Add resource tags.
5. Validate the template.
6. Deploy to a sandbox or review simulated deployment output.
7. Store the template in source control.
8. Add an IaC step to a pipeline.
9. Document how drift can occur.
10. Define review and approval rules for IaC changes.

Validation

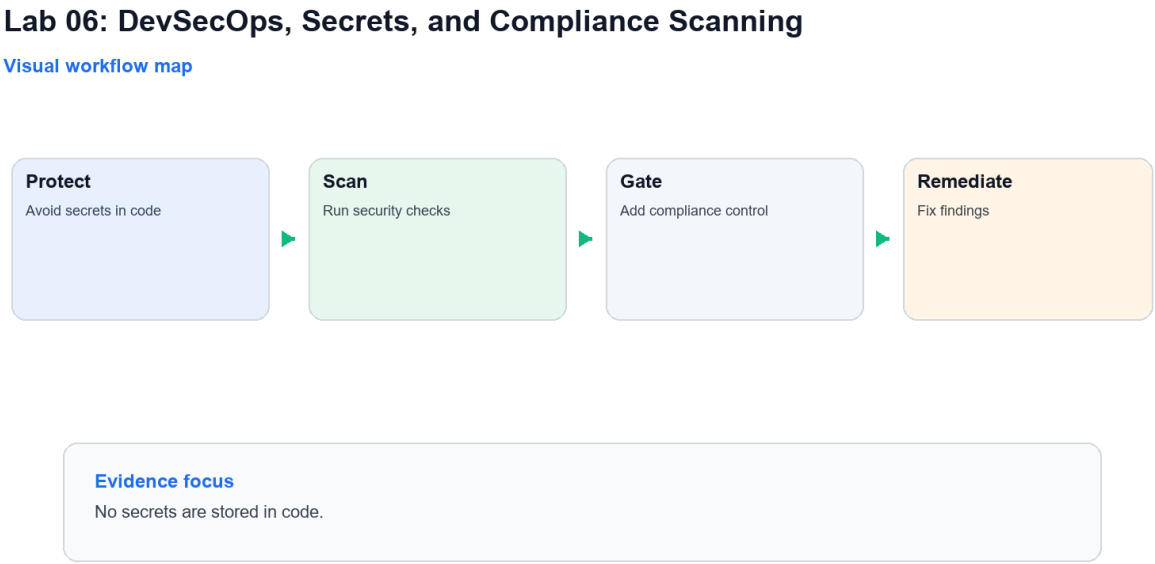
- Template is parameterized.
- Tags are included.
- Validation or deployment evidence is captured.
- Drift and review controls are documented.

Review Questions

1. Why use IaC?
2. What should be parameterized?
3. What is configuration drift?
4. Why should infrastructure changes use pull requests?

Lab 06 - DevSecOps, Secrets, and Compliance Scanning

Source: courseware/labs-source/lab-06-devsecops-secrets-compliance-scanning.md



What you will produce: Secrets → Scanning → Compliance Gate

Objectives

- Manage secrets safely.
- Design secure pipeline authentication.
- Configure scanning checks.
- Document compliance evidence.

Step-by-Step Guide

1. Identify secrets used by the sample app or pipeline.
2. Move secrets to GitHub secrets, Azure Pipelines variables, or Key Vault.
3. Compare service principals, managed identities, and OIDC.
4. Review service connections or GitHub Actions authentication.
5. Enable dependency scanning where available.
6. Enable secret scanning or add a secret scanning step.
7. Add CodeQL or static analysis conceptually or practically.
8. Review container scanning concepts.
9. Define a policy for critical findings.
10. Create a compliance evidence table.

Validation

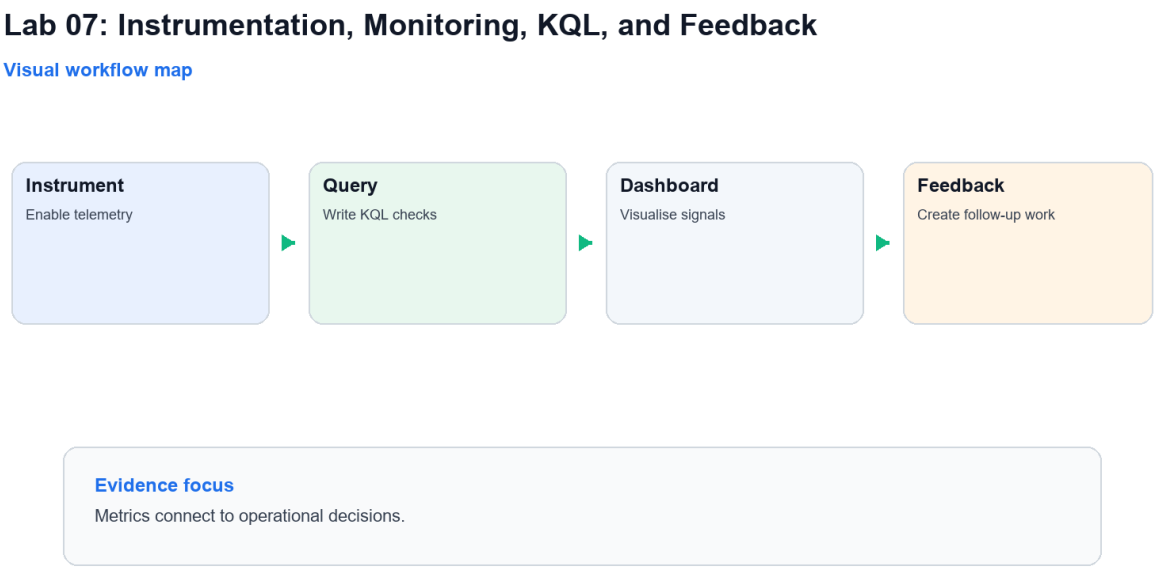
- No secrets are stored in code.
- Authentication design follows least privilege.
- Scanning plan covers code, dependencies, secrets, and containers.
- Evidence table is audit-friendly.

Review Questions

1. Why avoid long-lived secrets?
2. What is OIDC workload identity federation?
3. Why scan dependencies?
4. What should block a release?

Lab 07 - Instrumentation, Monitoring, KQL, and Feedback

Source: courseware/labs-source/lab-07-instrumentation-monitoring-kql-feedback.md



What you will produce: Telemetry → KQL → Dashboard → Feedback

Objectives

- Design monitoring strategy.
- Review Azure Monitor and Application Insights.
- Write KQL queries.
- Create feedback loops.

Step-by-Step Guide

1. Identify telemetry required for the sample application.
2. Define application, infrastructure, pipeline, security, and operations metrics.
3. Review Application Insights concepts.
4. Review Azure Monitor Logs concepts.
5. Create or design alert rules.
6. Review pipeline run history and failure rate.
7. Write simple KQL queries.
8. Identify distributed tracing concepts.
9. Define notifications to Teams, GitHub Issues, or Azure Boards.
10. Write an instrumentation strategy note.

Validation

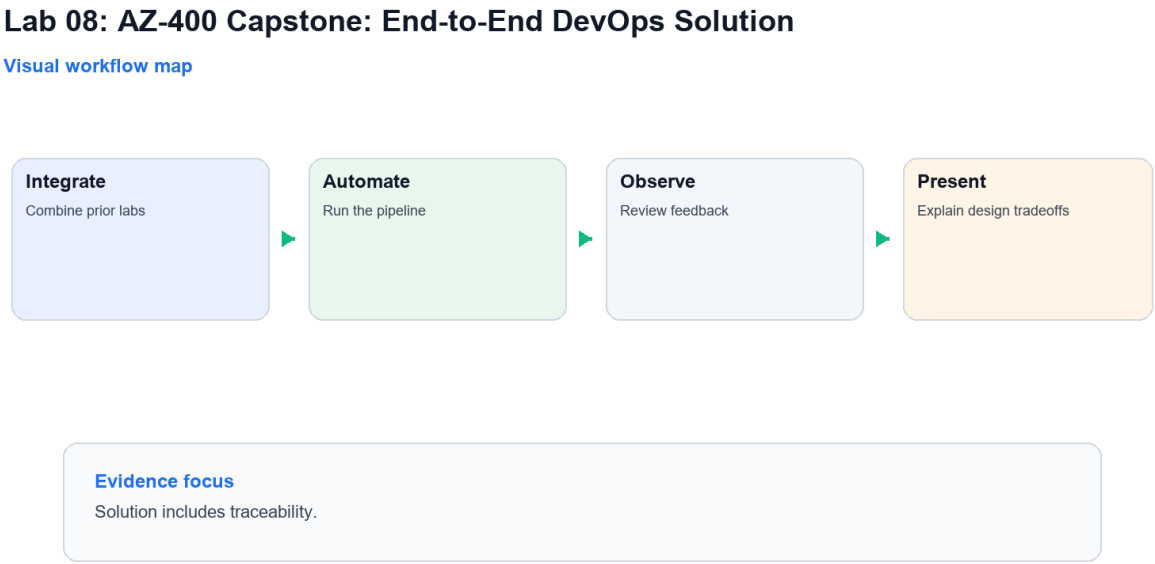
- Metrics connect to operational decisions.
- Alerts have clear action.
- KQL queries are documented.
- Feedback loop is practical.

Review Questions

1. What is MTTR?
2. Why collect pipeline metrics?
3. What is KQL used for?
4. How does feedback improve delivery?

Lab 08 - AZ-400 Capstone: End-to-End DevOps Solution

Source: courseware/labs-source/lab-08-capstone-az400-end-to-end-devops.md



What you will produce: Plan → Code → Build → Release → Monitor

Objectives

- Design a complete DevOps solution.
- Combine work tracking, source control, pipelines, security, and monitoring.
- Make tradeoff decisions.
- Build an exam study plan.

Step-by-Step Guide

1. Review the trainer-provided scenario.
2. Design flow of work using Boards or Issues.
3. Choose a branching and PR strategy.
4. Design CI pipeline stages.
5. Design deployment strategy and environment gates.
6. Add IaC and configuration management.
7. Add secrets, security scanning, and compliance evidence.
8. Add monitoring, alerts, and KQL.
9. Identify risks and mitigations.
10. Create an executive summary.
11. Create technical design notes.
12. Map weak areas to AZ-400 domains.
13. Write a 14-day study plan.

Validation

- Solution includes traceability.
- Pipeline strategy covers build, test, release, and rollback.
- Security and monitoring are built in.

- Study plan targets weak domains.

Review Questions

1. Which design tradeoff was hardest?
2. How does the solution improve deployment confidence?
3. What evidence would an auditor need?
4. Which AZ-400 domain needs more review?